4. Project Planning Phase

FitBuddy – AI Fitness Plan Generator using Gemini Models

Phase Report

Prepared from the supplied FitBuddy project documentation and organized into a project lifecycle deliverable.

1. Project Objective

Deliver a working local web application that generates personalized workout plans and tips, saves plans, supports feedback-based updates, and provides an admin overview.

2. Work Breakdown Structure

Task 1: confirm requirements and architecture. Task 2: create Python environment and dependencies. Task 3: implement database models and CRUD operations. Task 4: implement Gemini generation modules. Task 5: implement FastAPI routes. Task 6: create Jinja2 templates and CSS. Task 7: test end-to-end flows. Task 8: prepare documentation and demonstration.

3. Suggested Schedule

Day 1: requirements and project structure. Day 2: environment, configuration, and database. Day 3: AI generation and prompt handling. Day 4: routes and form validation. Day 5: templates and styling. Day 6: integration and testing. Day 7: documentation, screenshots, and demo rehearsal.

4. Dependencies

Python; FastAPI; Uvicorn; Jinja2; SQLAlchemy; python-multipart; python-dotenv; Google Generative AI SDK or the compatible Gemini SDK used by the implementation; ReportLab only if PDF export is added.

5. Risks and Mitigations

API quota or network failure: show a clear error and allow retry. Missing environment variables: validate configuration at startup. Invalid form values: use server-side validation. AI output formatting differences: request explicit structure and display as preformatted text. Sensitive data exposure: keep secrets in .env.

6. Deliverables

Source code; requirements.txt; .env.example; SQLite database created at runtime; HTML templates; CSS/static assets; test checklist; README setup guide; phase documentation; demo walkthrough.

7. Quality Gates

The application starts successfully; the homepage loads; plan generation and feedback update work with a valid API key; database persistence is verified; admin view displays records; common failures are handled clearly.

8. Planning Outcome

Implement the project in small, testable modules and verify each module before full integration.

Note: This document describes the intended design and workflow. Test outcomes should be recorded only after the application has been run and verified.